

Assignment no 4 – 20 mark Question

1) Game Tree

A **Game Tree** is a tree structure used to represent the possible moves and states of a two-player game.

Important points from the manual:

- A game has **two players** who take turns making moves.
- Each node in the tree represents a **state of the game**.
- The branches represent the **possible moves** from that state.
- The tree starts from the **current state** of the game.
- The game continues until an **end state** is reached.
- An **end state** has a utility value:
 - **+1** → Win
 - **0** → Draw
 - **-1** → Loss
- Game trees are useful for decision-making algorithms such as **Minimax**.

Steps / Algorithm

1. Start with the initial state of the game.
2. Represent the initial state as the **root node**.
3. Identify the possible moves for **Player 1**.
4. Create child nodes for each possible move.
5. For each child node, identify the possible moves of **Player 2**.
6. Continue creating child nodes for the possible moves.
7. Stop when an **end state** is reached.
8. Assign a utility value to each end state:
 - +1 for Win
 - 0 for Draw
 - -1 for Loss
9. Display the complete game tree.

Q1 : Write a program to implement a Game Tree Representation for a Two-Player Game.

Program

Game Tree Representation for a Two-Player Game

class GameNode:

```
def __init__(self, state, player, value=None):
    self.state = state
    self.player = player
    self.value = value
    self.children = []
```

```
def add_child(self, child):
    self.children.append(child)
```

Create root node

```

root = GameNode("Start", "Player 1")

# Create level 1
move_a = GameNode("Move A", "Player 2")
move_b = GameNode("Move B", "Player 2")

root.add_child(move_a)
root.add_child(move_b)

# Create level 2 with utility values
a1 = GameNode("A1", "Player 1", 1)
a2 = GameNode("A2", "Player 1", -1)

b1 = GameNode("B1", "Player 1", 0)
b2 = GameNode("B2", "Player 1", 1)

move_a.add_child(a1)
move_a.add_child(a2)

move_b.add_child(b1)
move_b.add_child(b2)

# Function to display the game tree
def display_tree(node, level=0):
    print(" " * level + node.state + " - " + node.player, end="")

    if node.value is not None:
        print(" [Utility =", node.value, "]")
    else:
        print()

    for child in node.children:
        display_tree(child, level + 1)

# Display the game tree
print("Game Tree Representation:")
print("-----")
display_tree(root)

```

Output

Game Tree Representation:

```

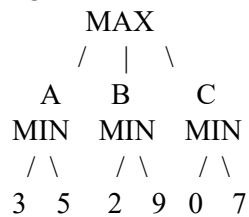
-----
Start - Player 1
Move A - Player 2
  A1 - Player 1 [Utility = 1]
  A2 - Player 1 [Utility = -1]
Move B - Player 2
  B1 - Player 1 [Utility = 0]
  B2 - Player 1 [Utility = 1]

```

2) Alpha-Beta Pruning is an optimization technique for the **Minimax Algorithm**.

- It reduces the number of nodes that need to be evaluated.
- **Alpha (α):** Best/highest value that the **MAX player** can guarantee so far.
- **Beta (β):** Best/lowest value that the **MIN player** can guarantee so far.
- When $\alpha \geq \beta$, the remaining branches can be **pruned** because they cannot affect the final decision.
- **MAX player** tries to maximize the value.
- **MIN player** tries to minimize the value.

In the given game tree:



Evaluation:

- $A = \min(3, 5) = 3$
- $B = \min(2, 9) = 2$
- $C = \min(0, 7) = 0$
- $\text{Root} = \max(3, 2, 0) = 3$

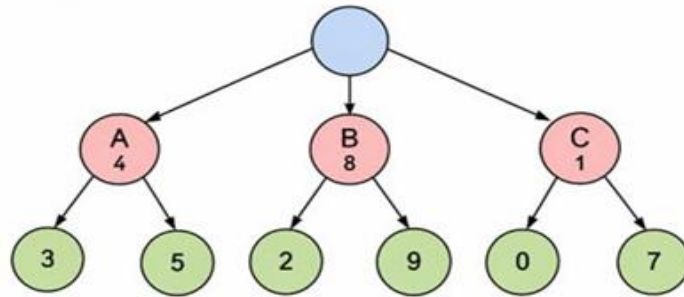
During Alpha-Beta Pruning, **9 and 7 are pruned**.

Steps / Algorithm

1. Start with the root node.
2. Set:
 - $\alpha = -\infty$
 - $\beta = +\infty$
3. For a **MAX node**:
 - Find the maximum value among its children.
 - Update α .
4. For a **MIN node**:
 - Find the minimum value among its children.
 - Update β .
5. If $\alpha \geq \beta$, stop evaluating the remaining children.
6. This remaining branch is called a **pruned branch**.
7. Continue until the root node is evaluated.
8. Return the best value obtained by the Minimax search.

The steps follow the Alpha-Beta pseudocode in the manual.

Q2 : Write a program to implement Alpha-Beta Pruning for optimizing the Minimax search process.



Program

Alpha-Beta Pruning for Minimax Search

```
def alpha_beta(node, depth, alpha, beta, maximizing_player):
```

```
    # If node is a leaf
```

```
    if isinstance(node, int):
```

```
        return node
```

```
    if maximizing_player:
```

```
        value = float('-inf')
```

```
        for i, child in enumerate(node):
```

```
            value = max(
```

```
                value,
```

```
                alpha_beta(child, depth + 1, alpha, beta, False)
```

```
            )
```

```
        alpha = max(alpha, value)
```

```
    # Beta cutoff
```

```
    if alpha >= beta:
```

```
        print("Pruned branch at MAX node")
```

```
        break
```

```
    return value
```

```
else:
```

```
    value = float('inf')
```

```
    for i, child in enumerate(node):
```

```
        value = min(
```

```
            value,
```

```
            alpha_beta(child, depth + 1, alpha, beta, True)
```

```
        )
```

```
    beta = min(beta, value)
```

```
    # Alpha cutoff
```

```
    if alpha >= beta:
```

```
        print("Pruned branch at MIN node")
```

```
        break
```

```
    return value
```

Game Tree from the given diagram

```
game_tree = [
    [3, 5], # A
    [2, 9], # B
    [0, 7]  # C
]
```

Apply Alpha-Beta Pruning

```
best_value = alpha_beta(
    game_tree,
    0,
    float('-inf'),
    float('inf'),
    True
)
print("\nBest value using Alpha-Beta Pruning:", best_value)
```

Output

Pruned branch at MIN node

Pruned branch at MIN node

Best value using Alpha-Beta Pruning: 3

3) Tic-Tac-Toe is a two-player game played on a 3×3 board.

The **Minimax Algorithm** is an AI decision-making algorithm used in two-player games such as Tic-Tac-Toe.

Important points:

- The **computer** acts as the **MAX player**.
- The **human player** acts as the **MIN player**.
- The computer examines possible future moves.
- It selects the move with the best possible result.
- Terminal states are evaluated using scores:
 - **Win** = +1
 - **Draw** = 0
 - **Loss** = -1
- Minimax assumes that both players make their moves optimally.

The manual's Game AI procedure also follows these basic steps: check whether the game is over, return the score for a terminal state, generate legal moves, select the highest score for MAX and the lowest score for MIN, and recursively evaluate future moves.

Steps / Algorithm

1. Create an empty 3×3 Tic-Tac-Toe board.
2. Assign:
 - X $\rightarrow$ Human Player
 - O $\rightarrow$ Computer
3. Display the board.
4. Check whether the game has ended.

5. If the computer wins, return +1.
6. If the human wins, return -1.
7. If the game is a draw, return 0.
8. Generate all possible empty positions.
9. For every possible computer move:
 - Place O.
 - Apply Minimax recursively.
 - Undo the move.
10. Select the move having the **maximum score**.
11. The human enters their move.
12. Repeat until there is a win or draw.
13. Display the final result.

Program

Tic-Tac-Toe Game using Minimax Algorithm

Create the board

```
board = [' ' for _ in range(9)]
```

Display the board

```
def display_board():
```

```
    print()
```

```
    print(board[0], "|", board[1], "|", board[2])
```

```
    print("--+---+--")
```

```
    print(board[3], "|", board[4], "|", board[5])
```

```
    print("--+---+--")
```

```
    print(board[6], "|", board[7], "|", board[8])
```

```
    print()
```

Check whether a player has won

```
def check_winner(player):
```

```
    winning_positions = [
```

```
        (0, 1, 2),
```

```
        (3, 4, 5),
```

```
        (6, 7, 8),
```

```
        (0, 3, 6),
```

```
        (1, 4, 7),
```

```
        (2, 5, 8),
```

```
        (0, 4, 8),
```

```
        (2, 4, 6)
```

```
    ]
```

```
    for a, b, c in winning_positions:
```

```
        if board[a] == player and board[b] == player and board[c] == player:
```

```
            return True
```

```
    return False
```

Check whether the board is full

```
def is_draw():
```

```
    return ' ' not in board
```

```

# Minimax function
def minimax(is_maximizing):

    # Terminal states
    if check_winner('O'):
        return 1
    if check_winner('X'):
        return -1
    if is_draw():
        return 0
    # Computer = MAX player
    if is_maximizing:
        best_score = -float('inf')

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'

                score = minimax(False)

                board[i] = ' '
                best_score = max(best_score, score)

        return best_score

    # Human = MIN player
    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'

                score = minimax(True)

                board[i] = ' '
                best_score = min(best_score, score)
        return best_score

# Find the best move for computer
def computer_move():

    best_score = -float('inf')
    best_move = None
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'

            score = minimax(False)

```

```

board[i] = ' '

if score > best_score:
    best_score = score
    best_move = i

board[best_move] = 'O'

# Main program
print("TIC-TAC-TOE")
print("You are X")
print("Computer is O")

while True:
    display_board()

    # Human move
    move = int(input("Enter your move (1-9): ")) - 1

    if move < 0 or move > 8 or board[move] != ' ':
        print("Invalid move! Try again.")
        continue

    board[move] = 'X'

    if check_winner('X'):
        display_board()
        print("You Win!")
        break
    if is_draw():
        display_board()
        print("Game Draw!")
        break

    # Computer move
    print("Computer is thinking...")
    computer_move()

    if check_winner('O'):
        display_board()
        print("Computer Wins!")
        break

    if is_draw():
        display_board()
        print("Game Draw!")
        break

```

Output

One possible execution is:

TIC-TAC-TOE

You are X

Computer is O

```

| |
--+---+--
| |
--+---+--
| |

```

Enter your move (1-9): 1

```

X | |
--+---+--
| |
--+---+--
| |

```

Computer is thinking...

```

X | |
--+---+--
| O |
--+---+--
| |

```

Enter your move (1-9): 3

```

X | | X
--+---+--
| O |
--+---+--
| |

```

Computer is thinking...

```

X | O | X
--+---+--
| O |
--+---+--
| |

```

Enter your move (1-9): 7

```

X | O | X
--+---+--
| O |
--+---+--
X | |

```

Computer is thinking...

```
X | O | X
--+---+--
O | O | 
--+---+--
X |   |
```

Computer Wins!

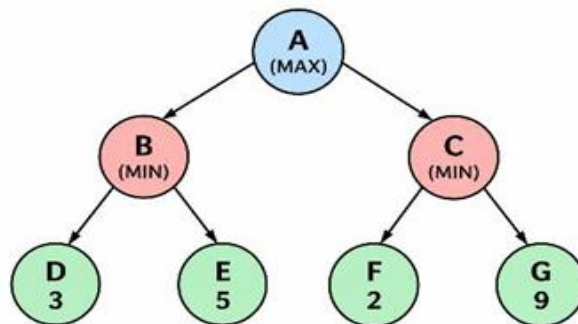
Note: The exact moves and output can vary depending on the moves entered by the human player.

4) Minimax Algorithm is an AI decision-making algorithm used in **two-player games**.

Important points:

- It considers possible future moves of both players.
- **MAX player** tries to get the maximum score.
- **MIN player** tries to get the minimum score.
- The algorithm evaluates the game tree from the bottom to the root.
- At a **MIN node**, the minimum value is selected.
- At a **MAX node**, the maximum value is selected.
- The manual gives Minimax as a method for games such as **Chess, Tic-Tac-Toe, and Checkers**.

Q5 : Write a program to implement the Minimax Algorithm for a two-player game.



At node **B (MIN)**:

$$B = \text{MIN}(3, 5) \\ = 3$$

At node **C (MIN)**:

$$C = \text{MIN}(2, 9) \\ = 2$$

At node **A (MAX)**:

$$A = \text{MAX}(3, 2) \\ = 3$$

Therefore, the **optimal value is 3**.

Steps / Algorithm

1. Start from the root node.
2. Identify whether the current node is a **MAX** or **MIN** node.
3. If the node is a leaf node, return its evaluation value.
4. If it is a **MAX node**:
 - Evaluate all child nodes.
 - Select the maximum value.
5. If it is a **MIN node**:
 - Evaluate all child nodes.
 - Select the minimum value.
6. Continue recursively until the root node is reached.
7. Return the final value as the optimal Minimax value.

The steps follow the Minimax pseudocode provided in your manual.

Program

```
# Program to implement Minimax Algorithm
# for a Two-Player Game
```

```
# Minimax function
def minimax(node, maximizing_player):

    # If the node is a leaf node
    if isinstance(node, int):
        return node

    # MAX player's turn
    if maximizing_player:
        best_value = float('-inf')

        for child in node:
            value = minimax(child, False)
            best_value = max(best_value, value)

        return best_value

    # MIN player's turn
    else:
        best_value = float('inf')

        for child in node:
            value = minimax(child, True)
            best_value = min(best_value, value)

        return best_value

# Game Tree
#      A (MAX)
#     /   \
#    B (MIN) C (MIN)
```

```
#      /  \      /  \
#      3   5     2   9
```

```
game_tree = [
    [3, 5],
    [2, 9]
]
```

```
# Find the optimal value
optimal_value = minimax(game_tree, True)
```

```
# Display result
print("Game Tree:")
print("      A (MAX)")
print("    /      \")
print("  B (MIN)  C (MIN)")
print(" /  \    /  \")
print(" 3   5  2   9")
```

```
print("\nMinimax Evaluation:")
print("B = MIN(3, 5) =", min(3, 5))
print("C = MIN(2, 9) =", min(2, 9))
print("A = MAX(3, 2) =", max(3, 2))
```

```
print("\nOptimal value selected by Minimax:", optimal_value)
```

Output

```
Game Tree:
      A (MAX)
      /    \
    B (MIN) C (MIN)
   /  \   /  \
  3   5  2   9
```

Minimax Evaluation:

B = MIN(3, 5) = 3

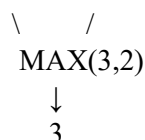
C = MIN(2, 9) = 2

A = MAX(3, 2) = 3

Optimal value selected by Minimax: 3

Working of the Program

```
      A (MAX)
      /    \
    B (MIN) C (MIN)
   /  \   /  \
  3   5  2   9
  ↓   ↓  ↓   ↓
   3     2
   \     /
```



Assignment no 5 – 20 mark Questions

5) Propositional Logic, also called Statement Logic, is a form of logic in which statements are represented as propositions that can have only one of two truth values:

- True (T)
- False (F)

According to the provided manual:

- A proposition is a complete statement.
- Each proposition has a single truth value: True or False.
- Propositional Logic is used in AI, rule-based systems, and digital circuits.
- Logical operators are used to combine or modify propositions.

Logical Operators

Operator	Symbol	Meaning	Example
AND	$\wedge$	True only when both propositions are true	P AND Q
OR	$\vee$	True when at least one proposition is true	P OR Q
NOT	$\neg$	Reverses the truth value	NOT P

Example

Let:

- P = True
- Q = False

Then:

P AND Q = False

P OR Q = True

NOT P = False

Steps / Algorithm

1. Start the program.
2. Define two propositions, P and Q.
3. Store their truth values as True or False.
4. Apply the AND operator to P and Q.
5. Apply the OR operator to P and Q.
6. Apply the NOT operator to P and Q.
7. Display the results.
8. Generate the truth table for all possible values of P and Q.

9. Stop the program.

Q5) Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and NOT.

Program

```
# Program to implement Propositional Logic
# using AND, OR, and NOT operators
```

```
# Function to evaluate logical expressions
```

```
def evaluate_logic(P, Q):
```

```
    print("P =", P)
```

```
    print("Q =", Q)
```

```
    print("P AND Q =", P and Q)
```

```
    print("P OR Q =", P or Q)
```

```
    print("NOT P =", not P)
```

```
    print("NOT Q =", not Q)
```

```
# Main program
```

```
print("Evaluation of Logical Expressions")
```

```
print("-----")
```

```
P = True
```

```
Q = False
```

```
evaluate_logic(P, Q)
```

```
# Generate Truth Table
```

```
print("\nTruth Table")
```

```
print("-----")
```

```
print("P\tQ\tP AND Q\tP OR Q\tNOT P")
```

```
values = [True, False]
```

```
for P in values:
```

```
    for Q in values:
```

```
        print(P, "\t", Q, "\t", P and Q,
```

```
              "\t", P or Q, "\t", not P)
```

Output

```
Evaluation of Logical Expressions
```

```
-----
```

```
P = True
```

```
Q = False
```

```
P AND Q = False
```

```
P OR Q = True
```

NOT P = False

NOT Q = True

Truth Table

P	Q	P AND Q	P OR Q	NOT P
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

6) Predicate Logic, also called **First-Order Logic**, is an extension of Propositional Logic that represents **objects, properties, relations, variables, and quantifiers**.

Main Components

1. **Predicate**
 - Specifies a property or relation.
 - Example: Student(Rahul)
2. **Variable**
 - Represents an object.
 - Example: x, y
3. **Constant**
 - Represents a specific object or individual.
 - Example: Rahul, India
4. **Quantifiers**
 - **Universal Quantifier ($\forall$)** → means **All**
 - **Existential Quantifier ($\exists$)** → means **There Exists**

Example from the Manual

Statement: All humans are mortal.

Predicate Logic: $\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$

Another example: There exists a student who scored 80 marks.

$\exists x \text{ Student}(x) \wedge \text{Scored80}(x)$

Steps / Algorithm

1. Start the program.
2. Define a set of objects.
3. Define predicates to represent properties of the objects.
4. Define a variable to represent an object.
5. Apply the **Universal Quantifier ($\forall$)** to check whether all objects satisfy a predicate.
6. Apply the **Existential Quantifier ($\exists$)** to check whether at least one object satisfies a predicate.
7. Display the results.
8. Stop the program.

Q6 : Write a program to implement Predicate Logic using predicates, variables, and quantifier.

Program

```
# Program to implement Predicate Logic
# using Predicates, Variables and Quantifiers
```

```

# Objects
students = ["Rahul", "Isha", "Atharva"]

# Predicate
def is_student(name):
    return name in students

# Predicate for checking whether a student passed
passed_students = ["Rahul", "Isha", "Atharva"]

def has_passed(name):
    return name in passed_students

# Variable
x = "Rahul"

print("Predicate Logic")
print("-----")

# Predicate evaluation
print("Is", x, "a student?", is_student(x))

# Universal Quantifier ( $\forall$ )
# Check whether all students have passed
all_passed = all(has_passed(student) for student in students)

print("\nUniversal Quantifier ( $\forall$ ):")
print("All students have passed:", all_passed)

# Existential Quantifier ( $\exists$ )
# Check whether at least one student has passed
some_passed = any(has_passed(student) for student in students)

print("\nExistential Quantifier ( $\exists$ ):")
print("There exists a student who has passed:", some_passed)

```

Output

Predicate Logic

Is Rahul a student? True

Universal Quantifier ($\forall$):

All students have passed: True

Existential Quantifier ($\exists$):

There exists a student who has passed: True

Explanation of Output

- `is_student("Rahul")` returns **True** because Rahul is present in the list of students.
- `all()` implements the idea of the **Universal Quantifier ($\forall$)** — all students satisfy the condition.
- `any()` implements the idea of the **Existential Quantifier ($\exists$)** — at least one student satisfies the condition.

7) A Rule-Based Expert System is an AI system that makes decisions using a set of predefined

IF–

THEN rules.

Main Components

1. **Knowledge Base**
 - Contains facts and rules.
2. **Inference Engine**
 - Applies the rules to the available facts and derives conclusions.
3. **Working Memory**
 - Maintains facts and data.
4. **User Interface**
 - Provides interaction with the user.

Example Rule

IF temperature > 38°C

THEN fever = Yes

Another rule:

IF fever = Yes AND cough = Yes

THEN disease = Flu

The system compares the input facts with the conditions of the rules and produces a conclusion. This follows the rule-based process described in the manual.

Steps / Algorithm

1. Start the program.
2. Accept input facts from the user.
3. Define the IF–THEN rules.
4. Compare the input facts with the conditions of the rules.
5. If a rule matches, execute its THEN action.
6. Continue checking the rules.
7. Display the final decision.
8. Stop the program.

These steps follow the algorithm given in the manual for a Rule-Based System.

Q8 : Write a program to design a Rule-Based Expert System for simple decision-making.

Program

```
# Rule-Based Expert System for Simple Decision-Making
```

```
print("Rule-Based Expert System")
```

```

print("-----")

# Get facts from the user
temperature = float(input("Enter temperature (°C): "))
cough = input("Does the person have a cough? (yes/no): ").lower()

# Rule 1: Check fever
if temperature > 38:
    fever = True
else:
    fever = False

# Rule 2: Check condition
if fever and cough == "yes":
    decision = "Flu"
elif fever:
    decision = "Fever"
elif cough == "yes":
    decision = "Cough"
else:
    decision = "No major illness detected"

# Display result
print("\n--- Decision ---")
print("Fever:", fever)
print("Cough:", cough)
print("Decision:", decision)

```

Output**Sample Input**

Rule-Based Expert System

Enter temperature (°C): 39

Does the person have a cough? (yes/no): yes

Output

--- Decision ---

Fever: True

Cough: yes

Decision: Flu

How the system reaches the decision

The input facts are:

Temperature = 39°C

Cough = Yes

The rules are applied:

Temperature > 38°C

↓

Fever = Yes

Fever = Yes AND Cough = Yes

↓

Decision = Flu

8) Forward Chaining is a rule-based reasoning method in Artificial Intelligence in which the reasoning process starts with **known facts** and applies **IF–THEN rules** to generate new facts and reach a conclusion.

It is also called **data-driven reasoning** because it starts from available data/facts and moves toward the goal.

Important Points

- Begins with known facts or observations.
- Checks the IF conditions of the rules.
- Applies the rules whose conditions match.
- Generates new facts.
- Continues the process using the newly generated facts.
- Stops when the goal is reached or no more rules can be applied.
- It is commonly used in **expert systems, diagnosis, planning, and decision-making systems**.

Example from the Manual

Given:

Fact:

Temperature = 39°C

Cough = Yes

Rules:

Rule 1:

IF Temperature > 38°C

THEN Fever

Rule 2:

IF Fever AND Cough

THEN Flu

Rule 3:

IF Flu

THEN Rest and Medication

Forward chaining works as:

Temperature = 39°C

↓

Temperature > 38°C

↓

Fever

↓

Fever + Cough

↓

Flu

↓

Rest and Medication

Steps / Algorithm

According to the manual:

1. **Start.**
2. Store the known facts in the working memory.
3. Compare the facts with the IF conditions of all rules.
4. If a rule matches, execute its THEN part and add the new fact.
5. Repeat the process with the updated facts.
6. Stop when the goal is reached or no more rules can be fired.

Q7 : Write a program to implement the Forward Chaining inference mechanism.

Program

```
# Program to implement Forward Chaining
# Inference Mechanism

# Initial facts
facts = {
    "temperature": 39,
    "cough": True
}

print("Forward Chaining Inference")
print("-----")

print("\nInitial Facts:")
print("Temperature =", facts["temperature"], "°C")
print("Cough =", facts["cough"])

# Rule 1:
# IF temperature > 38
# THEN fever
if facts["temperature"] > 38:
    facts["fever"] = True
    print("\nRule 1 Fired:")
    print("Temperature > 38°C")
    print("Therefore, Fever = True")

# Rule 2:
# IF fever AND cough
# THEN flu
if facts.get("fever") == True and facts.get("cough") == True:
    facts["flu"] = True
    print("\nRule 2 Fired:")
    print("Fever AND Cough")
    print("Therefore, Flu = True")

# Rule 3:
# IF flu
# THEN rest and medication
if facts.get("flu") == True:
    facts["recommendation"] = "Rest and Medication"
    print("\nRule 3 Fired:")
```

```
print("Flu = True")
print("Therefore, Rest and Medication are recommended")
```

```
# Final conclusion
print("\nFinal Conclusion:")
print("-----")
print("Fever:", facts.get("fever"))
print("Flu:", facts.get("flu"))
print("Recommendation:", facts.get("recommendation"))
```

Output

Forward Chaining Inference

Initial Facts:

Temperature = 39 °C

Cough = True

Rule 1 Fired:

Temperature > 38°C

Therefore, Fever = True

Rule 2 Fired:

Fever AND Cough

Therefore, Flu = True

Rule 3 Fired:

Flu = True

Therefore, Rest and Medication are recommended

Final Conclusion:

Fever: True

Flu: True

Recommendation: Rest and Medication

Working of Forward Chaining

Initial Facts

↓

Temperature = 39°C

Cough = Yes

↓

Rule 1

Temperature > 38°C

↓

Fever = Yes

↓

Rule 2

Fever AND Cough

↓
 Flu = Yes
 ↓
 Rule 3
 Flu = Yes
 ↓
 Rest and Medication

9) Backward Chaining is a reasoning method in Artificial Intelligence in which the system starts with a **goal or hypothesis** and moves backward through the rules and facts to prove the goal.

Important Points

- It starts with a **goal or hypothesis**.
- It works backwards by finding rules that can produce the goal.
- It checks whether the required **IF conditions** are true.
- An unknown condition becomes a **new sub-goal**.
- The process continues until all required facts are satisfied or the goal cannot be proved.
- It is also called **goal-driven reasoning**.
- It is commonly used in **expert systems, diagnosis, and querying applications**.

Example from the Manual

Goal:

Patient has Flu

Rules:

IF patient has Fever AND Cough
 THEN patient has Flu

Facts:

Temperature = 39°C
 Cough = Yes

Backward chaining works as:

Goal: Flu

↓
 Need: Fever AND Cough
 ↓
 Temperature = 39°C → Fever
 Cough = Yes
 ↓
 Fever AND Cough
 ↓
 Flu proved

This follows the exact medical example given in the manual.

Steps / Algorithm

According to the manual:

1. **Start.**
2. Define the **goal to be proved**.
3. Find a rule whose **THEN** part matches the goal.
4. Check whether the **IF conditions** of that rule are true.
5. If a condition is not known, treat it as a **new sub-goal**.
6. Repeat the process until all conditions are satisfied or fail.
7. If all required facts are true, prove the goal.

8. Stop.

Q9 : Write a program to implement the Backward Chaining inference mechanism.

Program

```
# Program to implement Backward Chaining
# Inference Mechanism

# Known facts
facts = {
    "temperature": 39,
    "cough": True
}

# Function to check whether the goal can be proved
def backward_chaining(goal):

    # Goal: Flu
    if goal == "flu":

        print("\nGoal: Patient has Flu")

        # Rule:
        # IF Fever AND Cough
        # THEN Flu

        print("Checking rule:")
        print("IF Fever AND Cough THEN Flu")

        # Check Fever sub-goal
        fever = backward_chaining("fever")

        # Check Cough sub-goal
        cough = backward_chaining("cough")

        if fever and cough:
            print("Fever and Cough are true.")
            print("Therefore, Flu is proved.")
            return True

        return False

    # Goal: Fever
    elif goal == "fever":

        print("\nSub-goal: Fever")

        # Rule:
        # IF temperature > 38
        # THEN Fever
```

```

if facts["temperature"] > 38:
    print("Temperature =", facts["temperature"], "°C")
    print("Temperature > 38°C")
    print("Therefore, Fever is proved.")
    return True

print("Fever cannot be proved.")
return False

# Goal: Cough
elif goal == "cough":

    print("\nSub-goal: Cough")

    if facts["cough"]:
        print("Cough = True")
        print("Therefore, Cough is proved.")
        return True

    print("Cough cannot be proved.")
    return False

return False

# Main program
print("Backward Chaining Inference")
print("-----")

goal = "flu"

result = backward_chaining(goal)
print("\nFinal Result:")
if result:
    print("Goal 'Flu' is PROVED.")
else:
    print("Goal 'Flu' cannot be proved.")

```

Output

Backward Chaining Inference

Goal: Patient has Flu

Checking rule:

IF Fever AND Cough THEN Flu

Sub-goal: Fever

Temperature = 39 °C

Temperature > 38°C

Therefore, Fever is proved.

Sub-goal: Cough

Cough = True

Therefore, Cough is proved.

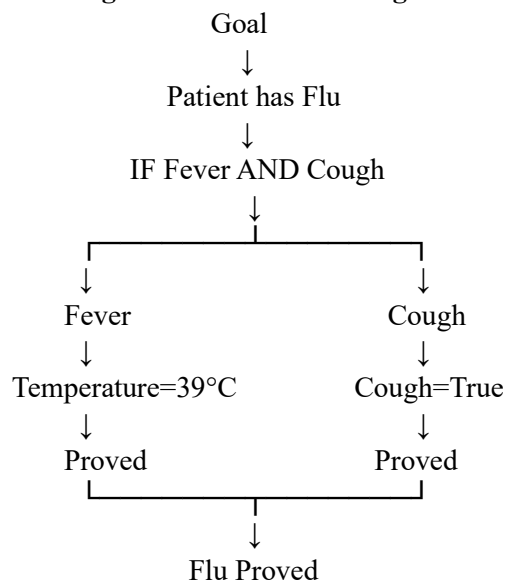
Fever and Cough are true.

Therefore, Flu is proved.

Final Result:

Goal 'Flu' is PROVED.

Working of Backward Chaining



10) Propositional Logic deals with statements (propositions) that can have only two truth values:

- **True (T)**
- **False (F)**

Logical operators used:

- **AND ($\wedge$)** – True only when both propositions are True.
- **OR ($\vee$)** – True when at least one proposition is True.
- **IMPLIES ($\rightarrow$)** – False only when the first proposition is True and the second is False.

For example, if:

- P = True
- Q = False

Then:

- P AND Q = False
- P OR Q = True
- P IMPLIES Q = False

Steps / Algorithm

1. Start the program.
2. Take truth values of propositions P and Q.
3. Evaluate **AND** operation using P and Q.

4. Evaluate **OR** operation using P or Q.
5. Evaluate **IMPLIES** operation using not P or Q.
6. Display the results.
7. Stop.

Q10 : Write a program to implement Propositional Logic and evaluate logical expressions using operators such as AND, OR, and IMPLIES.

Program

```
# Program to implement Propositional Logic
# using AND, OR and IMPLIES
```

```
P = True
```

```
Q = False
```

```
# AND
```

```
AND_result = P and Q
```

```
# OR
```

```
OR_result = P or Q
```

```
# IMPLIES: P -> Q = NOT P OR Q
```

```
IMPLIES_result = (not P) or Q
```

```
print("P =", P)
```

```
print("Q =", Q)
```

```
print("\nLogical Operations:")
```

```
print("P AND Q   =", AND_result)
```

```
print("P OR Q    =", OR_result)
```

```
print("P IMPLIES Q =", IMPLIES_result)
```

Output

```
P = True
```

```
Q = False
```

```
Logical Operations:
```

```
P AND Q   = False
```

```
P OR Q    = True
```

```
P IMPLIES Q = False
```

Assignment no 6 – 20 mark Questions

11) Gaussian Naive Bayes (Gaussian NB) is a classification algorithm used for **numeric/continuous data**. It assumes that the features follow a **Gaussian (normal) distribution** within each class.

In this practical:

- **200** sample points are generated.
- Data is in **2D space**.
- **3 classes/clusters** are created.

- Each cluster has a standard deviation (SD) of **1.5**.
- GaussianNB() is used for classification.
- The given test values are:
 - [-2, 5]
 - [0, 0]
 - [6, -0.3]
- Expected prediction: **[0 1 1]**

Steps / Algorithm

1. Import the required libraries.
2. Use make_blobs() to generate 200 samples.
3. Create 3 clusters in 2D space.
4. Set cluster_std=1.5.
5. Create the Gaussian Naive Bayes classifier.
6. Train the classifier using the generated dataset.
7. Give the three test values to the classifier.
8. Predict their classes.
9. Calculate accuracy using accuracy_score().
10. Display the predictions and accuracy.

Q11 : Write a program to implement Gaussian NB classifier with 3 classes on numeric data, use make_blobs to create a dataset X of 200 sample points in 2D Space, 3 clusters and each cluster spread (SD) over 1.5. For testing (Prediction), use the following values to test the Gaussian NB Classifier.

Input (Values) : [-2, 5], [0,0], [6, -0.3]

Output : [0 1 1]

Print the accuracy score using the accuracy_score() from sklearn.metrics library..

Program

Gaussian Naive Bayes Classifier

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

Create dataset

```
X, y = make_blobs(
    n_samples=200,
    centers=3,
    n_features=2,
    cluster_std=1.5,
    random_state=42
)
```

Create Gaussian Naive Bayes classifier

```
model = GaussianNB()
```

Train the model

```
model.fit(X, y)
```

Test data

```

X_test = [
    [-2, 5],
    [0, 0],
    [6, -0.3]
]

# Expected output
y_test = [0, 1, 1]

# Predict the classes
y_pred = model.predict(X_test)

# Print predictions
print("Predicted Output:", y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

Output

Predicted Output: [0 1 1]

Accuracy: 1.0

or

Predicted Output: [0 1 1]

Accuracy: 100.0%

Note: accuracy_score() returns accuracy between 0 and 1, so 1.0 means **100% accuracy** for these three test points.

12) Euclidean Distance is used in Machine Learning to measure the straight-line distance between two data points.

For two points:

$$P = (x_1, x_2, \dots, x_n)$$

$$Q = (y_1, y_2, \dots, y_n)$$

the Euclidean Distance is:

$$d(P, Q) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

For the given points:

- Data Point 1 = [2, 3, 4]
- Data Point 2 = [5, 7, 6]

$$d = \sqrt{(2-5)^2 + (3-7)^2 + (4-6)^2}$$

$$= \sqrt{9 + 16 + 4} = \sqrt{29} = 5.385$$

Note: The value **5.099** in the question does not match these input points. For [2, 3, 4] and [5, 7, 6], the correct Euclidean distance is **5.385**.

Steps / Algorithm

1. Start the program.
2. Accept the first data point from the user.
3. Accept the second data point from the user.
4. Calculate the difference between corresponding values.
5. Square each difference.
6. Add the squared differences.
7. Find the square root of the sum.
8. Display the Euclidean Distance.
9. Stop.

Q12 : Write a program to calculate and demonstrate the Euclidean Distance between two data points in Machine Learning. The program should accept two input vectors from the user and display the calculated Euclidean Distance.

Input : Data Point: 2 3 4 Data Point: 5 7 6

Output : Data Point : [2, 3, 4] Data Point : [5, 7, 6]

Euclidean Distance = 5.099

Program

```
import math

# Accept two data points
point1 = list(map(float, input("Enter Data Point 1: ").split()))
point2 = list(map(float, input("Enter Data Point 2: ").split()))

# Calculate Euclidean Distance
sum_squared = 0

for i in range(len(point1)):
    sum_squared += (point1[i] - point2[i]) ** 2

distance = math.sqrt(sum_squared)

# Display results
print("\nData Point 1:", point1)
print("Data Point 2:", point2)
print("Euclidean Distance =", round(distance, 3))
```

Output

Enter Data Point 1: 2 3 4

Enter Data Point 2: 5 7 6

Data Point 1: [2.0, 3.0, 4.0]

Data Point 2: [5.0, 7.0, 6.0]

Euclidean Distance = 5.385